

## Sematic Search Mini- project report

- In the Sematic Search Project, all-MiniLMTransformer model from sentence transformer library was used to convert the text from the files into embedding vector which is divided into chunks
- During the project, Pdf, txt and docx files were indexed from that text is converted into vector embedded which can be search later. The application does not support other format files.
- The document can be uploaded directly or if the user wants to save the file for longer periods the file can be saved inside the “documents” folder. Both methods are accepted in this project.

The pdf document is read using pyPDF2 reader which reads the document page by page, *extracts the text*, and stores in *text variable*, Python document (docx) read the DOCX files and txt files are read directly.

The extracted text file is split into words and then grouped into chunk of 300 words for further embedding.

- FAISS was used to calculate the similarity between the document text and the user's query.

In the project, the prepared text chunks are converted into embedding vectors using the encode () method of the Sentence Transformer model. The embeddings are returned as NumPy arrays and converted to the float32 data type. The embedding vectors are then L2-normalized and stored in the FAISS index.

When a user submits a query, the query goes through the same embedding process using the same model. The resulting query vector is normalized and passed to FAISS. FAISS compares the query vector with the stored document

vectors using the dot product. Since the vectors are L2-normalized, the dot product corresponds to cosine similarity, allowing FAISS to find the most similar text chunks.

- The search results are ranked based on dot products between L2 Normalized document chunks and query vector. The user can define the number of most relevant results using ***top\_k*** from UI. If the result of the dot product is closer to 1 means higher similarity, so the result will appear closer to the top of the ranking, while a lower value indicates lower similarity and will appear down the ranking.
- When the words in the queries are similar to the saved documents, or when the meaning of the queries is similar to the saved documents, the application provides better similarity results with higher scores.
- Queries that are very short such as single word, vague, or have meaning different from the saved documents can be difficult to for application.
- Sensitive documents were not used while testing. And developer owned documents and some scientific facts were used as source documents in pdf, doc and txt formats. Data quality was considered by using relevant, accurate, and clear documents to improve the quality of the search results.